Active Object

Season: 5 · Concurrency · Category: Concurrency · Level: Intermediate

Intent

- Method calls become async requests on a servant thread.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Personal assistant queues your tasks.

When to Use

- Concurrent / parallel workloads.
- The design force named in Intent shows up more than once.

When Not to Use

- Pure CPU-bound work already parallelized.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.concurrency.activeobject;

import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

/** Active Object — all calls run on a dedicated async servant thread. */
public final class ActiveObject implements AutoCloseable {
    private final ExecutorService servant = Executors.newSingleThreadExecutor(r -> {
        var t = new Thread(r, "active-object");
        t.setDaemon(true);
        return t;
    });
    private int value;

    public Future<Integer> add(int delta) {
        return servant.submit(() -> { value += delta; return value; });
    }

    public Future<Integer> get() {
        return servant.submit((Callable<Integer>) () -> value);
    }

    @Override public void close() { servant.shutdownNow(); }
}
```

Modern Java 25 / Functional

- Executors.newVirtualThreadPerTaskExecutor() is the modern default for blocking tasks.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: concurrency/activeobject/
```

Pitfalls

- Prefer java.util.concurrent and virtual threads over custom pools.
- Always interrupt/shutdown cleanly.

Interview Q&A

- Q: Virtual threads vs platform threads? A: Virtual threads excel at blocking I/O concurrency; still bound by CPU for pure compute.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Method calls become async requests on a servant thread.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Personal assistant queues your tasks. [0:40] Intent — State the design force in one sentence: Method...